

ETHIOPIAN PAY-LATER PLATFORM

Technical Architecture & Development Proposal

Production-oriented technical proposal; no application code

Addis Ababa-first • Provider-independent

Document Code	DOC-06
Version	1.0
Date	21 August 2026
Prepared For	Software Architects • Developers • DevOps • QA • Security
Prepared By	Eleanor Tefera s(Fintech)

CONFIDENTIAL WORKING DOCUMENT

Document Control

Version	Date	Prepared By	Status	Summary
1.0	21 August 2026	Eleanor Tefera(Fintech)	Working baseline	Initial researched package release

Research Status Legend

VERIFIED	Directly supported by an official/primary source.
REPORTED	Supported by a reputable third-party source.
INFERENCE	Reasonable conclusion from available evidence.
ASSUMPTION	Design assumption for planning; not presented as market fact.
REQUIRES CONFIRMATION	Needs provider, bank, NBE, legal or business-partner confirmation.

Table of Contents

Document Control.....	i
Research Status Legend	i
1. Technical Executive Summary	1
2. Architecture Decision	1
3. High-Level Architecture.....	1
4. Technology Stack.....	1
5. Core Modules	1
6. Money Representation	2
7. Concurrency and Idempotency.....	2
8. Payment Provider Layer	2
9. Customer Mobile Application	2
10. Merchant Mobile Application	2
11. Admin Web Platform	3
12. Enterprise Merchant Connection	3
13. Security Architecture.....	3
14. Reliability / Observability	3
15. Testing Strategy	3
16. Technical Risks	4
17. Final Technical Recommendation	4
References	i

1. Technical Executive Summary

Recommend a modular monolith for the first production platform: one deployable backend with strongly separated business modules, PostgreSQL as the authoritative relational database, Redis for cache/locks/queues, Flutter for customer and merchant applications, and a modern web admin. This minimizes early operational complexity while preserving module boundaries that can later be extracted if scale/team ownership justifies it.

2. Architecture Decision

Option	Benefit	Risk	Recommendation
Modular monolith	Fast delivery, transactional consistency, easier testing/operations.	Requires discipline to preserve module boundaries.	SELECT for MVP/pilot.
Service-based	Independent deployment for selected domains.	Distributed failure/observability complexity.	Use later for clearly justified provider/reporting workloads.
Microservices	Independent scale/team ownership.	High operational and financial consistency complexity.	Do not start here.

3. High-Level Architecture

Customer Flutter App + Merchant Flutter App + Admin Web + Enterprise Merchants → API/Application Layer → Identity/KYC | Customer/Merchant | Credit/Risk | Order/BNPL | Installments | Payments | Ledger/Settlement | Refunds/Disputes | Notifications/Support → PostgreSQL + Redis/Queues + Secure Object Storage → External Credit/KYC/Payment/Notification Partners

4. Technology Stack

Layer	Recommendation	Why	Alternative / When
Backend	Laravel / current supported PHP	Strong productivity, validation/auth/queues/scheduler/ecosystem; suitable for modular monolith and transactional workflows.	Java/.NET/Node can work; change only for team/enterprise constraints, not fashion.
Customer/Merchant Mobile	Flutter	One codebase for Android-first Ethiopia and iOS; strong UI control and offline caching.	Native when device/payment SDK constraints force it.
Admin	React/Next.js or Laravel + React	Rich data-heavy admin and role-based workflows.	Server-rendered Laravel views for smaller internal MVP if speed matters.
Database	PostgreSQL	Strong constraints, transactional behavior, NUMERIC, indexes and JSON where justified.	MySQL acceptable if team standard, but PostgreSQL preferred for financial domain rigor.
Cache/Locks/Queues	Redis + Laravel queues	Rate limits, short-lived cache, distributed locks, worker queues.	Managed queue service at larger scale/ops maturity.
Object Storage	S3-compatible private bucket	KYC/docs/evidence with encryption, lifecycle and signed URLs.	Provider-specific secure object storage.
Notifications	SMS + FCM/APNs + email	OTP and financial reminders/alerts.	Multiple SMS providers for resilience later.
Deployment	Containerized workloads on reputable cloud/managed services	Repeatable deploys, managed DB/backup, observability.	VMs only if operational/regulatory constraints require them.
CI/CD	Git-based pipeline with gated production approval	Automated quality with human approval for sensitive releases.	Equivalent enterprise CI platform.

5. Core Modules

Module	Responsibilities
Identity & Access	Accounts, login, OTP, roles, sessions, devices.

KYC/KYB	Customer/merchant verification, documents, manual review.
Customer & Merchant	Profiles, branches, employees, merchant status.
Credit & Risk	First-time eligibility, limits, reservations, transaction decisions, offers, fraud signals.
Orders & BNPL	Sales, checkout, plan selection, customer acceptance.
Installments	Schedules, due dates, repayment allocation, overdue states.
Payments	Provider-independent attempts, confirmation, retries, webhooks/polling.
Ledger & Settlement	Double-entry postings, merchant payable, settlement batches/items.
Refunds/Disputes	Full/partial refunds, evidence, adjustments.
Reconciliation	Provider/bank/internal matching and exceptions.
Notifications/Support	SMS/push/email, tickets and complaint SLA.
Audit/Configuration	Immutable audits and versioned business/provider settings.

6. Money Representation

Never use binary floating-point for authoritative money. Store ETB amounts as integer minor units (cents/santim equivalent where product policy uses them) or PostgreSQL NUMERIC with a strict rounding policy. Every API and ledger contract must define currency and scale.

7. Concurrency and Idempotency

- Reserve credit exposure atomically before purchase activation.
- Use unique idempotency keys on checkout, payments, refunds and settlements.
- Use database unique constraints for provider references and event IDs.
- Process webhook events through an inbox table; use outbox pattern for reliable event publishing where needed.
- Use row/version locks for concurrent limit reservations and settlement posting.

8. Payment Provider Layer

PaymentService → PaymentProviderInterface → TelebirrAdapter | CbeBirrAdapter | MpesaEthiopiaAdapter | BankAdapter | GatewayAdapter

- Core business modules work with internal payment commands/states, not provider payloads.
- Adapters translate internal requests and provider responses.
- Adapters declare capability flags: initiate, query, refund, mandate, sandbox, webhook, settlement info.
- Unavailable/unconfirmed capabilities remain disabled in production configuration.

9. Customer Mobile Application

- Secure onboarding, KYC and first-time eligibility.
- Home with approved/available amount and next due installment.
- Merchant discovery/QR/checkout and transparent plan disclosure.
- Payment initiation with explicit pending/reconciliation UI.
- History, support, notifications, security/devices.
- Local cache for read-only data; financial/credit actions remain server-authoritative.

10. Merchant Mobile Application

- KYB/onboarding, branches and staff.
- Quick Sale and dynamic QR.

- Transaction state that explicitly separates APPROVED from WAITING_FOR_FIRST_PAYMENT/PENDING.
- Refund requests and settlement reports.
- Restricted employee permissions and activity audit.

11. Admin Web Platform

- Customer/KYC/KYB reviews
- Credit decisions/limits/manual review
- Payment/reconciliation exceptions
- Refund/settlement operations with separation of duties
- Fraud/disputes/support complaints
- Configuration/version publishing
- Audit/reporting/export with role filtering

12. Enterprise Merchant Connection

Expose versioned REST APIs/webhooks for merchant order creation, quote/eligibility, checkout, status, cancellation/refund and reconciliation exports. Enterprise credentials are scoped to merchant/branches; every write uses idempotency keys and signed/authenticated webhooks.

13. Security Architecture

- MFA for privileged staff; strong password hashing and optional customer PIN/biometric local unlock.
- Short-lived access tokens/session rotation; revoke lost devices.
- TLS everywhere; database/object encryption; field-level encryption for especially sensitive attributes.
- Central secrets manager and key rotation.
- Webhook signature verification when available plus nonce/timestamp/replay prevention.
- WAF/rate limits/bot protection; fraud events from anomalous auth/payment behavior.
- Admin break-glass access logged and reviewed.

14. Reliability / Observability

- Structured logs with correlation IDs, no secrets/PII leakage.
- Metrics for API latency, errors, queue depth, provider success/pending rates, reconciliation breaks and settlement failures.
- Distributed tracing for checkout/payment where practical.
- Health/circuit breaker per external provider.
- Dead-letter queue review and replay runbook.

15. Testing Strategy

- Unit tests for money, schedules, eligibility rules and state machines.
- Integration tests with database/Redis/provider simulators.
- Contract tests against provider sandbox/documentation when available.
- E2E tests for customer + merchant + payment + settlement.
- Security/performance/DR testing before production.
- Replay tests for duplicate/delayed callbacks and concurrent purchases.

16. Technical Risks

Risk	Technical Response
Provider capability changes	Adapter/capability flags; contract tests.
Credit model changes	Configurable creditor/funder and policy versions.
Duplicate callbacks	Inbox/idempotency/unique provider refs.
Race on credit limit	Atomic reservations/locking.
Ledger mismatch	Balanced postings + reconciliation.
Low connectivity	Small payloads, resumable UX, server-authoritative states.
Premature scale complexity	Modular monolith; extract services only with measured need.

17. Final Technical Recommendation

Build a small but financially rigorous modular platform. Complexity should be concentrated in correctness—credit decisions, payment state, ledger, reconciliation, security and audit—not in premature microservices.

References

[ETH-NBE-01] VERIFIED — National Bank of Ethiopia. Payment Instrument Issuers / System Operators.

<https://nbe.gov.et/payment-instrument-issuers-system-operators/>

Supports: Current licensed/commercialized payment instrument issuers and payment system operators, including Telebirr, Kacha, M-PESA Ethiopia, EthSwitch, ArifPay, Chapa and SantimPay.

[ETH-NBE-02] VERIFIED — National Bank of Ethiopia. Financial Stability Report — March 2026.

https://nbe.gov.et/wp-content/uploads/2026/03/Financial-Stability-Report_MARCH_2026.pdf

Supports: Current payment-sector regulatory framework, digital-finance context and systemic risk context.

[ETH-NBE-03] VERIFIED — National Bank of Ethiopia. National Digital Payments Strategy 2026–2030 — Draft.

https://nbe.gov.et/wp-content/uploads/2025/12/Ethiopia_NDPS_Draft_F.pdf

Supports: Interoperability direction and digital-payment strategy context; document is a draft.

[ETH-NBE-04] VERIFIED — National Bank of Ethiopia. Financial Consumer Protection — Complaint Handling.

<https://nbe.gov.et/fcpe/complaint/>

Supports: Complaint escalation after first contacting the financial service provider; NBE review where unresolved/no response within 10 business days.

[ETH-DP-01] VERIFIED — Federal Democratic Republic of Ethiopia / Ministry of Justice. Personal Data Protection Proclamation No. 1321/2024.

<https://justice.gov.et/en/law/personal-data-protection-proclamation/>

Supports: Ethiopian personal-data protection framework.

[ETH-QR-01] VERIFIED — EthSwitch. Interoperable QR Standard.

<https://ethswitch.com/wp-content/uploads/2024/12/Interoperable-QR-Standard.pdf>

Supports: Interoperable QR design direction for merchant payments and limitations of closed-loop QR.

[TEL-01] VERIFIED — Ethio telecom. Telebirr Developer Portal — Getting Started.

<https://developer.ethiotelcom.et/docs/GettingStarted>

Supports: Publicly documented Telebirr business integration categories including C2B, B2B, H5, Mini App and subscription payment.

[TEL-02] VERIFIED — Ethio telecom. Telebirr H5 C2B Web Payment Integration.

<https://developer.ethiotelcom.et/docs/category/h5-c2b-web-payment-integration>

Supports: Public create-order, checkout, query/status, callback and refund integration documentation.

[TEL-03] VERIFIED — Ethio telecom. Telebirr Schedule Payment / CreateDisburseOrder.

<https://developer.ethiotelcom.et/docs/Schedule%20Payment/CreateDisburseOrder>

Supports: Public documentation for mandate-based scheduled deduction; BNPL repayment use requires commercial/regulatory confirmation.

[CBE-01] VERIFIED — Commercial Bank of Ethiopia. CBE Birr.

<https://combanketh.et/ways-of-banking/cbe-birr>

Supports: Official description of CBE Birr customer/mobile-money services.

[CBE-02] VERIFIED — Commercial Bank of Ethiopia. CBE Products and Services.

https://combanketh.et/cbeapi/uploads/CBE_Products_and_services_English_84e511c740.pdf

Supports: CBE Birr supports transfers, cash out, airtime, goods purchase and bill payment; no equivalent public direct BNPL integration specification verified.

[MPE-01] VERIFIED — Safaricom Ethiopia. M-PESA Ethiopia Merchant Terms / Business Use.

<https://www.safaricom.et/>

Supports: M-PESA Ethiopia supports merchant payments/business arrangements; exact BNPL API onboarding and sandbox require provider confirmation.

[QTR-TC] VERIFIED — PayLater Qatar. PayLater General Terms and Conditions — client-provided 20-page copy.

Client-provided PDF reviewed 21 August 2026

Supports: Customer/merchant/PayLater roles, eligibility, 25% first payment, PayMore, limits, repayment, default, restructuring, disputes, data handling and account rules.

[QTR-PL-01] VERIFIED — PayLater Qatar. PayLater Terms and Conditions.

<https://paylaterapp.com/terms-and-conditions/>

Supports: Current public PayLater Qatar contractual product model.

[QTR-PL-02] VERIFIED — PayLater Qatar. How can I sign up for PayLater?.

<https://help.paylaterapp.com/en/support/solutions/articles/154000139292-how-can-i-sign-up-for-paylater->

Supports: Phone OTP, QID capture and liveness-based customer onboarding.

[QTR-PL-03] VERIFIED — PayLater Qatar. What is my credit limit and how much can I spend?.

<https://help.paylaterapp.com/en/support/solutions/articles/154000139303-what-is-my-credit-limit-and-how-much-can-i-spend-with-paylater->

Supports: Credit limit uses creditworthiness, PayLater repayment history and account usage.

[QTR-PL-04] VERIFIED — PayLater Qatar. Where can I use PayLater?.

<https://help.paylaterapp.com/en/support/solutions/articles/154000139291-where-can-i-use-paylater->

Supports: Use across online and in-store merchants.

[QTR-CBQ-01] VERIFIED — Commercial Bank Qatar / Vodafone Qatar. Vodafone Easy-Buy / CBQ Buy Now Pay Later.

<https://www.vodafone.qa/en/buynowpaylater>

Supports: Bank-card installment model for online and selected physical stores; full purchase reduces card limit and customer repays equal monthly installments.

[QTR-VF-01] VERIFIED — Vodafone Qatar. Vodafone Partners with Commercial Bank to Offer BNPL.

<https://www.vodafone.qa/en/about-us/media/press-release/vodafone-partners-with-commercial-bank-to-offer-buy-now-pay-later-installment-plan>

Supports: Qatar device-installment model through bank partnership and merchant stores.